DevOps Engineer – Technical Exercise

Overview: Today you'll take a small web service from code to a running deployment on a local Kubernetes cluster, the GitOps way. You have **90 minutes** of build time, then a 45-minute review with us. Work through the parts in order; finishing everything isn't expected. We care more about how you work and what you understand than about how far you get.

What you get: a MacBook with internet access and admin rights. Nothing else is installed; choosing and setting up your tools is part of the test.

Part 0: Set up and plan

1. Install whatever you need to build containers and run Kubernetes locally, plus Terraform, kubectl, git and an editor.
2. Log in to GitHub with your own account and create a new repository for today.
3. Before building, start a `README.md` with your plan: the steps you'll take and the tools you chose. Keep updating it as you go and commit it along with your work.
4. Write a tiny web service in any language. `GET /` returns an **HTML page** (not JSON) that shows, readable in the browser, a message, the app version and the name of the pod serving it. `GET /healthz` returns 200.
5. Add a Dockerfile; the image runs as a non-root user. Build and run it locally to check it works.

Part 1: Cluster as code

1. Terraform creates a local Kubernetes cluster (1 control-plane + 2 worker nodes) and installs Argo CD and an ingress controller.
2. `terraform apply` brings everything up; `terraform destroy` removes it.

Part 2: GitOps deployment

1. The app's Kubernetes manifests live in your repo, and Argo CD syncs them.
2. At least 2 replicas, readiness and liveness probes, and resource requests and limits.
3. The app is exposed through an Ingress and opens in the browser on this laptop.

Part 3 (good to have): CI/CD

1. A GitHub Actions workflow, on push to `main`, builds the image and pushes it to GitHub Container Registry tagged with the commit SHA.
2. The workflow updates the image tag in git; Argo CD rolls it out. No `kubectl apply` from CI.
3. **Zero-downtime:** change the app's message, push, and show that a loop of requests sees no failed responses during the rollout.

Bonus (only if time is left)

- A Horizontal Pod Autoscaler (min 2, max 5)
- Prometheus and Grafana installed through Argo CD
- In the README: what you'd change to run this on EKS in production

Guidelines

- Use any documentation, Google, Stack Overflow and any AI assistant (with your own account).
- **AI coding agents (Claude Code, Cursor, Copilot, OpenCode, etc.) are allowed.** Read every prompt you send and every change the agent makes before you accept it. You need to understand what it did and why: in the review we'll ask you to explain any file or line in your repo, and "the AI wrote it" isn't an answer.
- Save the prompts you give the AI as plain text in your repo (for example `ai-prompts.md`) and commit them together with the changes they produced.
- Keep your AI chat or agent session open; we may look at it together in the review.
- The interviewer answers questions about the requirements but won't help with the solution.
- Commit as you go; we read the history.

At the end of build time

1. Push everything to the repo, including your AI prompts file, and leave the cluster running.
2. Finish your `README.md` with: what you finished, what's missing, one decision you made and why, and **one place where the AI was wrong and how you noticed**.
3. Take a 10-minute break while we look at your repo, then we'll do the review together at this laptop.